CSE 4345: Big Data Analytics

Week 4, Part 1 — The Key-Value Paradigm: MapReduce, Pig, Hive & HBase

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Course & Lecture Information

Lecture Identity

Course:	CSE 4345
Week / Part:	4 of 11 | Part 1 of 2
CLO:	CLO1
PLO:	PLO(a)

CLO1

"Understand big data characteristics and the technologies used to process them"

Course Progression

Week 2: Data integration challenges
Week 3: Where data lives (HDFS)
Week 4: How data is *processed* (MapReduce +

Part 1 Covers

- Why we need a new computation model for HDFS
- The MapReduce programming model (Map · Shuffle/Sort · Reduce)
- The Combiner: local mini-reducer optimisation
- The Partitioner: directing keys to Reducers
- Full MapReduce execution pipeline with fault tolerance
- Apache Pig: dataflow scripting (Pig Latin)
- Apache Hive: SQL-on-Hadoop (HiveQL)
- Apache HBase: column-family NoSQL on Hadoop
- Ecosystem tool selection matrix

Lecture Agenda

Why a New Computation Model?

The Computation Problem on Distributed Storage

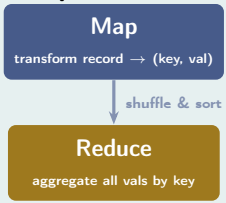
The Challenge (2003)

Google has petabytes of web crawl data on a 1,000-node GFS cluster. To build a search index, they need to:

- Process every document (sequential scan over petabytes)
- Aggregate data by key (e.g., group URLs by domain)
- Do this **fast**: a single thread processes ∼100 MB/s; a petabyte takes >2.7 hours per pass even at full disk speed
- **Tolerate failures**: any node can fail mid-job; the job must still complete

The Key Insight (Dean & Ghemawat, 2004)

Every large-scale computation can be expressed as:



"If you can write it as Map + Reduce, the framework parallelises it automatically across 1,000 machines."

The SQL Answer is Wrong Here

The MapReduce Programming Model

MapReduce: The Word Count Canonical Example

Problem

Count the frequency of every word in a multi-terabyte document collection.

Step-by-Step Execution

Input split (one per Mapper):
`"the cat sat on the mat"` **Map phase** — emit (word, 1) for each word:

`(the,1) (cat,1) (sat,1) (on,1) (the,1) (mat,1)`

Shuffle & Sort — group by key:

`(cat,[1]) (mat,[1]) (on,[1]) (sat,[1]) (the,[1,1])`

Reduce phase — sum each list:

Formal Definition

Map: $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$
Reduce: $(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_3)$

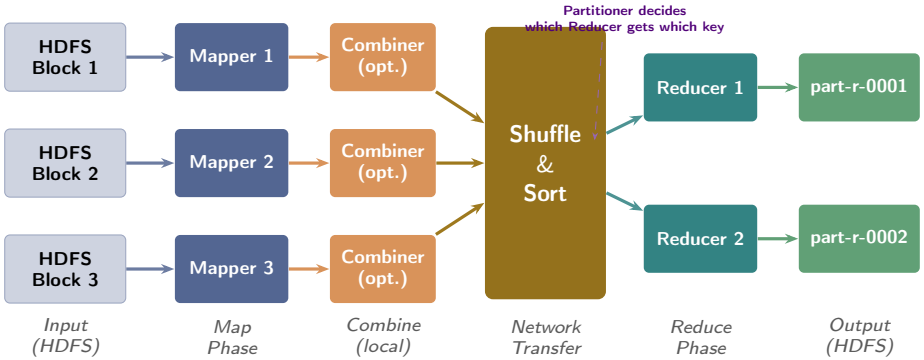
Both functions are **user-defined**; the framework handles everything else: data distribution, task scheduling, progress tracking, and failure recovery.

Mapper Pseudocode

```
def map(key, value):
    # value = one line of text
    for word in value.split():
        emit(word, 1)
```

Reducer Pseudocode

MapReduce Execution Pipeline



The Shuffle is the Bottleneck

The shuffle phase transfers intermediate data **across the network**. Minimizing shuffle volume is the primary MapReduce performance optimisation. The **Combiner** reduces shuffle traffic by pre-aggregating locally.

The Combiner: Local Mini-Reducer

Why the Combiner Exists

In word count, each Mapper emits $(the, 1)$ for every occurrence of "the" in its input split. If a split contains 10,000 occurrences of "the", the Mapper emits 10,000 pairs, each sent over the network during shuffle.

With a **Combiner** running on the Mapper's output *before* network transfer:

- Locally sum all $(the, 1)$ pairs $\rightarrow$ emit $(the, 10000)$
- **Network traffic reduced by $\sim$10,000$\times$** for this key

Combiner Constraint

The Combiner must have the **same input and**

Without vs. With Combiner

Without:

- Mapper output: `(the,1)` $\times$ 10,000 pairs
- Network send: 10,000 records per key

With Combiner:

- Mapper output: `(the,1)` $\times$ 10,000
- Combiner locally: `(the,10000)` — 1 record
- Network send: **1 record per key**

Java: Setting Combiner

```
Job job = Job.getInstance(conf);
job.setMapperClass(WordMapper.class);
// Combiner class = Reducer class
job.setCombinerClass(SumReducer.class);
job.setReducerClass(SumReducer.class);
```

MapReduce Fault Tolerance

How MapReduce Handles Node Failure

The **ApplicationMaster** (YARN) monitors every task. If a node fails:

1. ApplicationMaster detects missing heartbeat (>10 minutes = task dead)
2. **Failed Map tasks** are re-scheduled on any available node (input data still in HDFS)
3. **Failed Reduce tasks** are re-scheduled; must re-read shuffle data
4. **Speculative execution**: if a task is unusually slow, run a duplicate on another node — accept the first to finish, kill the other ("straggler mitigation")

Bangladesh Context: Pathao Ride Analytics

Scenario: Pathao processes ∼500,000 completed rides per day. Each ride record (JSON, ∼2 KB) contains: rider ID, driver ID, start/end coordinates, distance, fare, timestamp. **MapReduce jobs run nightly:**

- Mapper: emit `(driver_id, fare)`
- Reducer: compute driver earnings per day
- Combiner: pre-sum fares within each Mapper's split

Scale: ∼1 GB/day; runs across 10 nodes in <3 min. During a node crash mid-job, the failed Mapper tasks re-execute on healthy nodes; the job still completes.

Apache Pig: Dataflow Scripting

Apache Pig: ETL in Pig Latin

What is Pig?

Apache Pig is a **high-level dataflow scripting platform** that compiles a scripting language called **Pig Latin** into MapReduce jobs automatically. Designed for data engineers who need to build complex ETL pipelines without writing Java.

- **Procedural**: expresses *how* data flows step-by-step
- Each statement defines a transformation on a **relation** (bag of tuples)
- Optimizer rewrites the script into an efficient MapReduce DAG
- Extensible: custom UDFs in Java, Python, or Ruby

Pig Latin: bKash Transaction Example

Find top-5 merchants by total revenue

```
-- Load raw transactions from HDFS
txns = LOAD '/data/bkash/txns'
       USING PigStorage(',')
       AS (txn_id:chararray,
           merchant:chararray,
           amount:float,
           status:chararray,
           dt:chararray);

-- Filter: only completed payments
paid = FILTER txns
       BY status == 'COMPLETED';

-- Group by merchant
grp = GROUP paid BY merchant;

-- Sum revenue per merchant
rev = FOREACH grp GENERATE
      group          AS merchant,
      SUM(paid.amount) AS revenue;
```

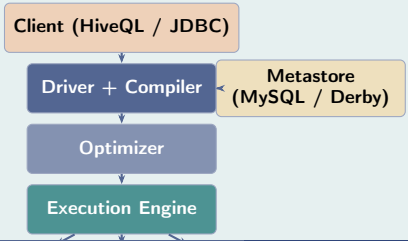
Apache Hive: SQL on Hadoop

Apache Hive: HiveQL and the Metastore

What is Hive?

Apache Hive provides a **data warehousing layer on top of HDFS**, exposing a SQL dialect called **HiveQL** that is compiled into MapReduce, Tez, or Spark jobs at runtime. Data analysts write SQL; the cluster does the rest.

Hive Architecture



Hive Metastore

The Metastore is a **relational database** (MySQL, Derby) that stores:

- Table schemas (column names, types)
- Partition metadata
- Location of HDFS data for each table
- Statistics for query optimisation

Hive uses "schema-on-read": data files in HDFS are untyped; schema is applied only at query time.

When to Use Hive

- Analysts who know SQL and don't want to write MapReduce
- Ad-hoc queries over large, structured datasets

HiveQL in Practice

HiveQL: Create Table and Query

Analyzing NID registration data (hypothetical)

```
-- Create external table over existing HDFS data
CREATE EXTERNAL TABLE nid_registrations (
  nid_no       STRING,
  district     STRING,
  upazila      STRING,
  gender       STRING,
  birth_year   INT,
  reg_date     STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/data/nid/2024/';

-- Find districts with most registrations
-- by age group (under-30 vs 30+)
SELECT
  district,
  CASE WHEN (2024 - birth_year) < 30
       THEN 'under_30'
       ELSE '30_and_above'
```

Key Hive Features

- **Partitioning**: store data in subdirectories by partition key (e.g., `/data/dt=2024-01-15/`) to skip irrelevant data at query time
- **Bucketing**: hash rows into fixed-size files within a partition; enables efficient joins and sampling
- **ORC/Parquet**: columnar file formats that dramatically reduce I/O for analytical queries (covered in Week 5)
- **UDF**: User-Defined Functions in Java for custom analytics

Hive vs. Traditional Warehouse

Apache HBase: Column-Family NoSQL

HBase: The Open-Source Bigtable

What is HBase?

Apache HBase is a **distributed, column-family NoSQL database** that runs on top of HDFS. It is the open-source implementation of Google's Bigtable (Chang et al., 2006). **Formal definition (Bigtable paper):**

> *"A Bigtable is a sparse, distributed, persistent multidimensional sorted map. The map is indexed by a row key, column key, and a timestamp."*

Contrast with HDFS/Hive/MapReduce:

- HDFS + MapReduce/Hive: **write once, read many** (batch)
- HBase: **random read/write** in milliseconds (OLTP-like)

HBase Column-Family Data Model

Row Key	CF: personal_info		CF: scores	
	name	*city*	*math*	*sci*
user001	Alice	Dhaka	95	88
user002	Bob	*null*	72	*null*
user003	*null*	Ctg	*null*	91

Null cells consume no storage (sparse)

HBase Architecture and Row Key Design

HBase Core Components

- **HMaster**: coordinates RegionServers; handles schema changes (analogous to NameNode for metadata)
- **RegionServer**: serves reads/writes for a set of **Regions** (horizontal table partitions by row key range)
- **Region**: contiguous range of rows, stored as **HFiles** on HDFS
- **ZooKeeper**: distributed coordination; stores HBase master address; detects RegionServer failures
- **WAL (Write-Ahead Log)**: all writes go to WAL on HDFS first, then to in-memory **MemStore**, then flushed to HFile

Row Key Design: Critical for Performance

In HBase, the **row key determines physical data order**. Poor key design causes:

- **Region hotspotting**: sequential keys (e.g., timestamps) send all writes to one Region Server
- Solution: **salt** the key (prepend hash prefix), **reverse** the timestamp, or use a **composite key**

Row key examples:

Bad	Good
`20240101:user1`	`7a3b:user1:20240101`
`20240101:user2`	`3f1c:user2:20240101`
(hotspot: same Region)	(distributed: hash prefix

Ecosystem Tool Selection

Choosing the Right Tool: Decision Matrix

Requirement	Raw MapReduce	Pig	Hive	HBase
Interface language	Java API	Pig Latin (script)	HiveQL (SQL)	Java API / Shell
User profile	Java developer	Data engineer	SQL analyst	Java developer
Access pattern	Sequential batch	Sequential batch	Sequential batch	Random access
Query latency	Minutes	Minutes	Minutes	Milliseconds
Data structure	Any	Semi-structured	Tabular	Sparse / wide
Schema requirement	None	Loose	On-read	Column families
Write pattern	Write-once	Write-once	Write-once	Read/write

Ivy League Alignment

MapReduce in Top University Curricula

Stanford CS246 (Leskovec) — Mining of Massive Datasets

MapReduce is the **computational substrate** for the entire course:

- **Chapter 2**: treats MapReduce as a programming model with specific algebraic properties: *commutativity* and *associativity* are required for Combiners; students prove why average cannot use a Combiner but sum can
- **Cost model**: algorithm analysis uses number of **reducer inputs** as the primary complexity measure, replacing time complexity — this drives algorithm design
- **Matrix multiplication** and **graph algorithms** (PageRank) are shown to map onto MapReduce through clever key design

Key exam concept at Stanford:
"Design a 2-pass MapReduce algorithm for the join of two 1

MIT 6.830 — Database Systems

MIT compares MapReduce with parallel databases:

- DeWitt & Stonebraker (2008): "*MapReduce: A major step backwards*" — critiques lack of schema, indexes, and query optimiser
- Pavlo et al. (2009) **benchmark**: MapReduce vs. parallel RDBMS — finds RDBMS 10–100× faster for structured queries, but MapReduce wins on **fault tolerance and unstructured data**

Hive and Pig were created directly in response to these critiques — adding a query layer and optimiser on top of MapReduce.

Student Assessment Pool

Assessment—MCQ Set A

CLO1

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

Q1. In the MapReduce model, which phase sorts and groups intermediate key-value pairs **before** passing them to Reducers?

- Map (b) **Shuffle and Sort** (c) Combine (d) Partition

Q2. Which Hadoop ecosystem tool uses a scripting language called **"Pig Latin"**?

- Hive (b) **Pig** (c) HBase (d) Sqoop

Instructor Notes

Q1: Students often confuse *Combine* with *Shuffle and Sort*. The Combiner is *optional* and runs before shuffle; the Shuffle and Sort phase is *mandatory* and runs as part of the framework — it is what delivers sorted key groups to each Reducer. **Q2:** Students sometimes choose Hive because it is more commonly named in general usage. Reinforce: Hive uses **HiveQL** (SQL); Pig uses **Pig Latin** (dataflow scripting).

Assessment—MCQ Set B

CLO1

Q3. HBase is the open-source implementation of which Google system?

(a) MapReduce (b) GFS (c) **Bigtable** (d) Dremel

Q4. Which of the following best describes the HBase data model?

(a) A relational table with primary and foreign key constraints
(b) A document store where each row contains a JSON blob
(c) **A sparse, distributed, persistent multidimensional sorted map indexed by row key, column key, and timestamp**
(d) A graph database storing nodes and edges in adjacency lists

Instructor Notes

Q3: Dremel (option d) is the precursor to Google BigQuery — a common distractor. The correct mapping is: GFS → HDFS; Bigtable → HBase; MapReduce → MapReduce; Dremel → BigQuery. **Q4:** This quote is taken verbatim from the Bigtable paper (Chang et al., 2006) and is the definition students should memorise. Option (a) is the most common wrong answer from students familiar only with relational databases.

Assessment—True / False

CLO1

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. MapReduce was originally published by Facebook in 2004.	**False**
2. Hive queries are compiled at runtime into MapReduce, Tez, or Spark jobs and run on HDFS data.	**True**
3. Pig is better suited than Hive for SQL-fluent data analysts who want to write ad-hoc queries.	**False**
4. HBase supports low-latency random read/write operations, unlike HDFS, which is optimised for sequential reads.	**True**

Model Justifications

Assessment—Descriptive Questions *CLO1*

Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

D1. Walk through the full MapReduce execution pipeline for the **word count** problem. Identify the roles of the Mapper, Combiner, Shuffle/Sort, and Reducer. Include a **diagram** showing data flow between these phases.

D2. Compare **Apache Pig** and **Apache Hive**. For each tool, state: (a) the language users write in, (b) the programming model (procedural vs. declarative), and (c) one type of task it is better suited for.

D3. Explain the HBase **column-family data model**. How does it differ from a relational schema? Give a concrete example of a table with two column families and at least two rows, including a **sparse** row.

Assessment—Analytical Questions *CLO1*

Instructions

Answer in **200–300 words** with step-by-step reasoning. Marks awarded for correct process, not just conclusion.

A1. MapReduce Algorithm Design:
You have a 500 GB log file containing Pathao ride records. Each record contains: `(driver_id, rider_id, distance_km, fare_BDT, timestamp)`. Design a MapReduce job to find the **top-10 drivers by total fare earned in January 2024**.

- Write pseudocode for the Map function (include the filter condition)
- Write pseudocode for the Reduce function
- Can a Combiner be used? If yes, write its pseudocode. If no, explain why.
- How would you find the global top-10 if there are 20 Reducer partitions?

A2. Tool Selection for a Real Scenario:
A Dhaka-based fintech startup uses HDFS to store 3 years of bKash transaction records (2 TB). They need: (i) nightly batch reports aggregating total transaction volume by district (SQL-fluent finance team), and (ii) real-time fraud lookups by transaction ID (<10 ms response). Recommend a specific tool for each requirement. Justify your choices on four dimensions: access pattern,

Textbook & Reading References

Primary Textbooks for Week 4

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Ch. 2, 12, 20**
  - Ch. 2: MapReduce · Ch. 12: Hive · Ch. 20: HBase
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 4, 5**
  - Ch. 4: MapReduce · Ch. 5: Pig and Hive
- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. **Ch. 2**
  - Ch. 2: MapReduce and the New Software Stack (cost model, algorithm design)

Supplementary (Covered in Week 4, Part 2 — Seminal Papers)

- Dean, J. & Ghemawat, S. (2004). **MapReduce: Simplified Data Processing on Large Clusters.** *OSDI 2004*, pp. 137–150. **[PRIMARY SEMINAL PAPER]** (>20,000 citations)
- Chang, F. et al. (2006). **Bigtable: A Distributed Storage System for Structured Data.** *OSDI 2006*, pp. 205–218. **[HBase origin paper]**
- Thusoo, A. et al. (2009). **Hive — A Warehousing Solution Over a Map-Reduce Framework.** *VLDB 2009*. **[Hive origin paper, Facebook]**

Questions & Discussion

Week 4, Part 1 | CSE 4345 Big Data Analytics

"MapReduce is simple, but its simplicity is deceptive. The real insight is not the model itself, but the realisation that nearly every large-scale data processing problem can be cast into it."

— Jeff Dean, Google (OSDI 2004 presentation)

Next Lecture: Week 4, Part 2

Seminal Paper 1: Dean & Ghemawat (2004) — *"MapReduce: Simplified Data Processing on Large Clusters"*

Seminal Paper 2: Chang et al. (2006) — *"Bigtable: A Distributed Storage System for Structured Data"*

Case Study: Facebook's data warehouse migration to Hive (2009)
Scale: 2 PB data · 1,000+ engineers · from Java to SQL democratisation

Reading before Part 2: [TW] Ch. 2 · Dean & Ghemawat (2004) paper (see references)